

CSS — переменные, Grid, адаптивность и движение

custom properties, media queries, transform, transition и animation

Максим

старший разработчик · Яндекс Еда

<CSS>

tokens.css

VAR

```
:root {  
  --accent: #7c3aed;  
  --gap: 20px;  
}
```

GRID



@MEDIA

48rem



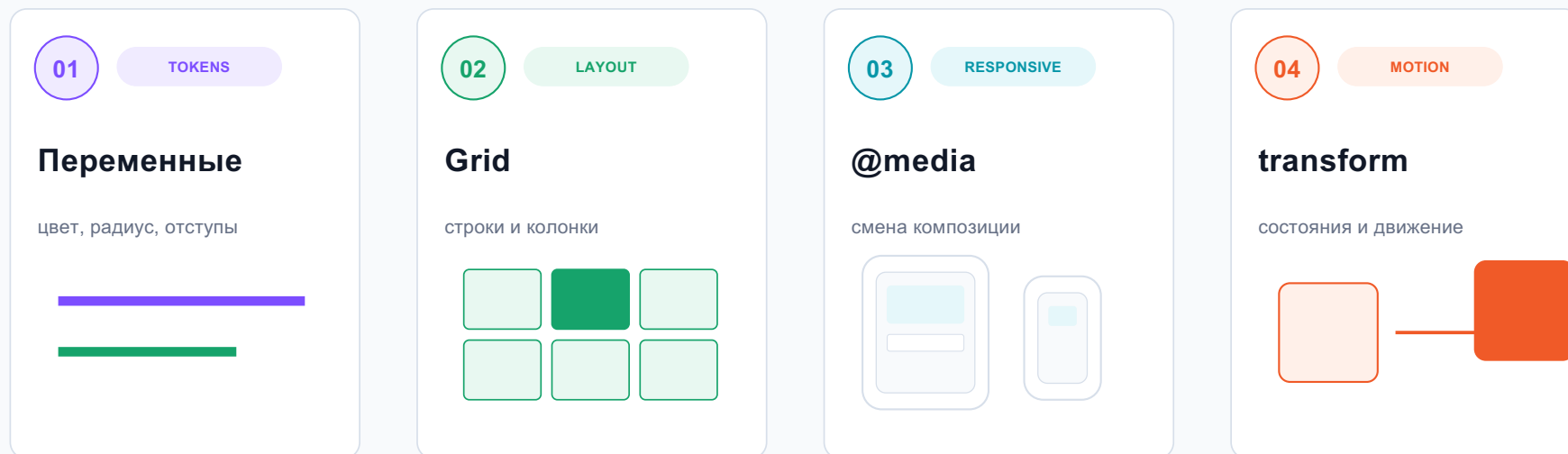
transform + transition

Что успеем за 90 минут

01	CSS-переменные и токены	СИСТЕМА	0–15 мин
02	Grid: двумерная раскладка	СЕТКА	15–45 мин
03	Медиа-запросы и адаптивность	КОНТЕКСТ	45–65 мин
04	Transform и плавные переходы	ДВИЖЕНИЕ	65–80 мин
05	@keyframes, reduced motion и практика	АНИМАЦИЯ	80–90 мин

По ходу — два коротких разбора в DevTools и одна итоговая мини-практика.

Один компонент — четыре механизма CSS



Сквозной пример: адаптивный блок тарифов без дублирования и «магических» значений.

Повторяющиеся значения превращаются в «магические числа»

```
styles.css

.button {
  background: #7c3aed;
  padding: 12px 20px;
  border-radius: 16px;
}

.card {
  border-color: #7c3aed;
  gap: 20px;
  border-radius: 16px;
}

.badge { color: #7c3aed; }
```

Изменить тему = найти все совпадения

одинаковые числа могут иметь разный смысл

#7c3aed — что это?

бренд · акцент · фокус · ошибка?

Именуем роль, а не значение

--color-accent · --radius-card · --space-4

Пользовательское свойство + функция var()

tokens.css

```
:root {
  --color-accent: #7c3aed;
  --radius-card: 16px;
  --space-4: 16px;
}

.button {
  background: var(--color-accent);
  border-radius: var(--radius-card);
  padding: var(--space-4);
}
```

`--color-accent: #7c3aed;`

ИМЯ

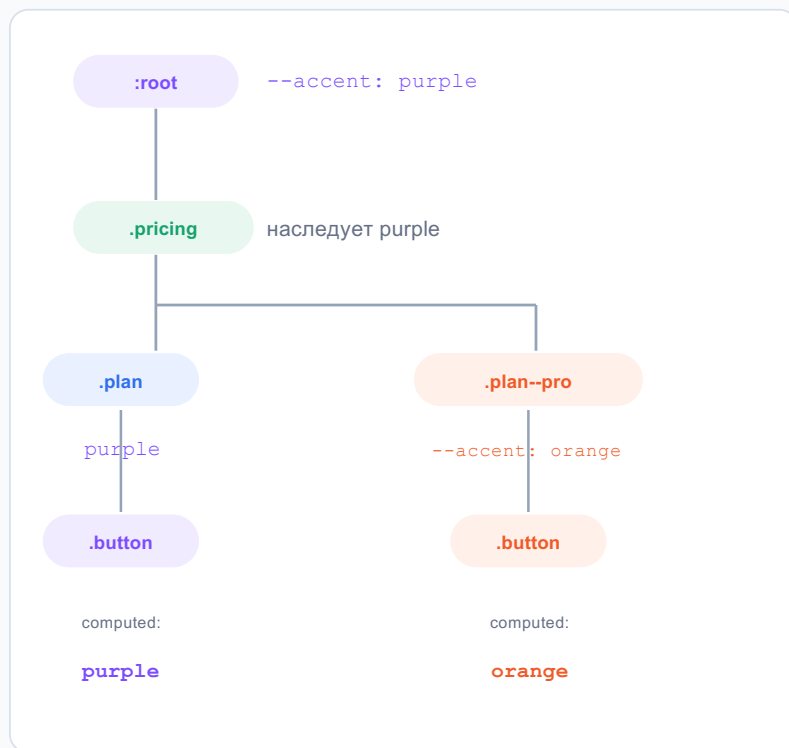
ЗНАЧЕНИЕ

начинается с --

почти любые CSS-
токены`var(--color-accent)`

- ✓ объявляем как обычное свойство
- ✓ используем в значениях других свойств
- ✓ участвует в каскаде и наследовании

Область видимости строится по DOM-дереву



```
theme.css

:root { --accent: #7c3aed; }

.pricing {
  --gap: 20px;
}

.plan--pro {
  --accent: #f05a28;
}

.button { background: var(--accent); }
```

Ближайшее значение по дереву побеждает

локальная тема компонента без новых селекторов

Fallback помогает не во всех ошибках

FALLBACK РАБОТАЕТ

```
ok.css

.link {
  color: var(--link-color, #2f6fed);
}
```



#2f6fed

переменная отсутствует → берём запасное значение

FALLBACK НЕ СПАСАЕТ

```
invalid.css

:root { --brand: 20px; }

.title {
  color: var(--brand, red);
}
```



color invalid

переменная определена, но итоговое значение не подходит color

Токены отделяют дизайн-систему от компонентов

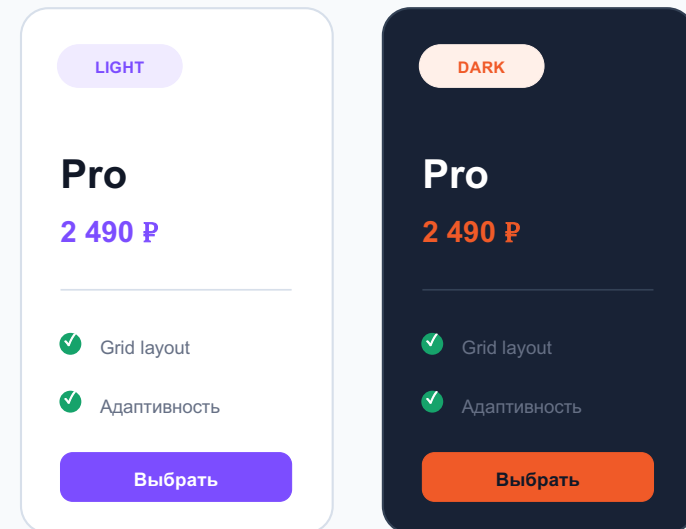
```
theme.css

:root {
  --surface: #ffffff;
  --text: #111827;
  --accent: #7c3aed;
  --radius-card: 18px;
}

[data-theme="dark"] {
  --surface: #182235;
  --text: #f8fafc;
}

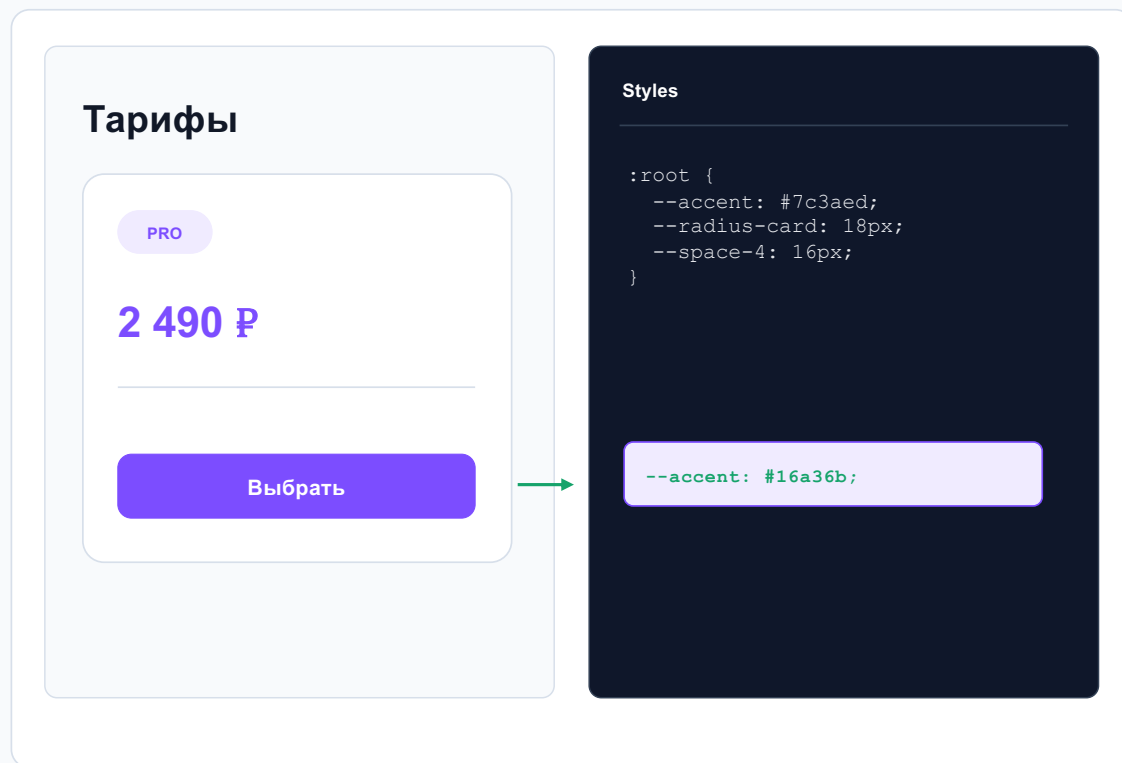
.plan {
  background: var(--surface);
  color: var(--text);
  border-radius: var(--radius-card);
}
```

Один компонент — две темы



Токены лучше называть по назначению: `--surface`, а не `--white`.

Меняем тему через DevTools за три минуты



ЗАДАНИЕ

Измените три токена

- ✓ акцентный цвет
- ✓ радиус карточки
- ✓ шаг отступов

Наблюдайте: сколько элементов изменилось без правки компонентов?

3 МИН

Flexbox управляет осью, Grid — плоскостью

FLEXBOX

Одно измерение

распределяем элементы вдоль строки или колонки

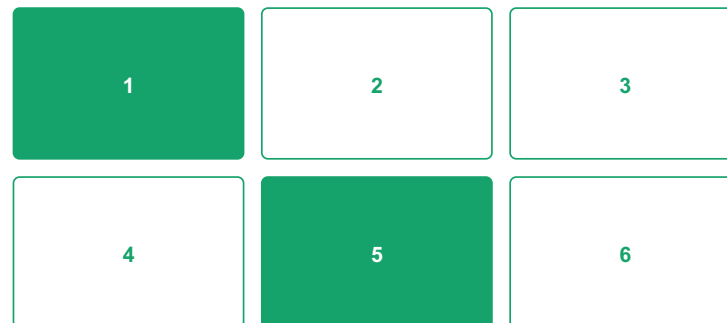


главная ось

GRID

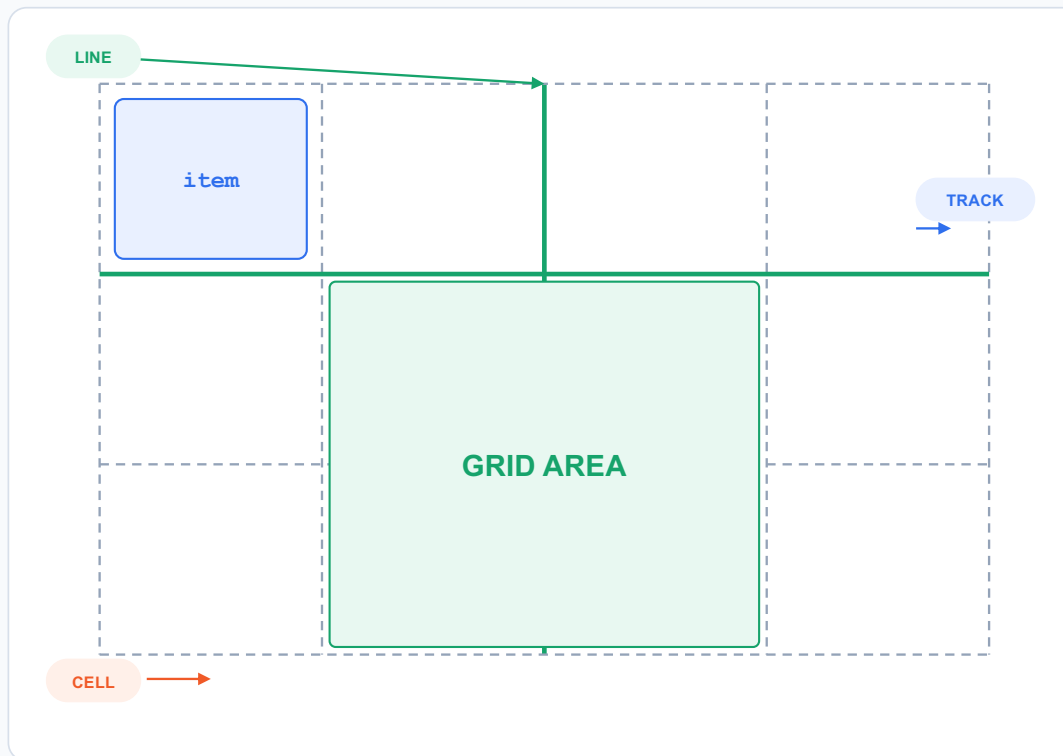
Два измерения

строки и колонки участвуют в раскладке одновременно



Они дополняют друг друга: Grid для макета, Flexbox для внутреннего выравнивания.

Сетка состоит из линий, треков, ячеек и областей



CONTAINER элемент с display: grid

ITEM прямой ребёнок контейнера

LINE граница строки или колонки

TRACK пространство между двумя линиями

CELL пересечение строки и колонки

AREA одна или несколько ячеек

grid-template задаёт явные строки и колонки

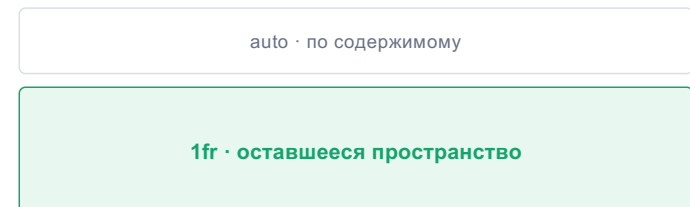
layout.css

```
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr 2fr;  
  grid-template-rows: auto 1fr auto;  
  min-height: 100vh;  
  gap: 20px;  
}
```

Колонки



Строки



fr делит свободное пространство после фиксированных и контентных размеров.

repeat() и minmax() описывают правила, а не пиксели

ПОВТОР

```
repeat(3, 1fr)
```



вместо 1fr 1fr 1fr

ГРАНИЦЫ

```
minmax(220px, 1fr)
```

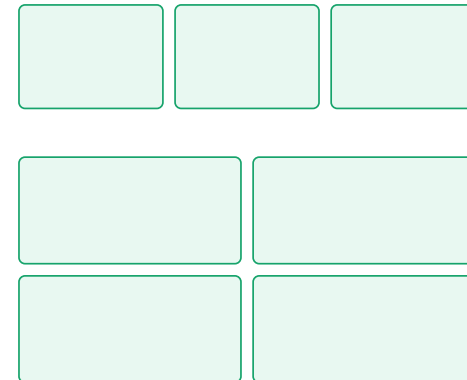


минимум 220px

может расти до 1fr

КОМБИНАЦИЯ

```
repeat(auto-fit,  
minmax(220px, 1fr))
```



сетка сама меняет число колонок

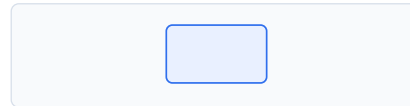
gap отделяет треки, alignment — содержимое и всю сетку

grid.css

```
.grid {  
  display: grid;  
  gap: 16px 24px;  
  
  justify-items: center;  
  align-items: start;  
  
  justify-content: center;  
  align-content: start;  
}
```

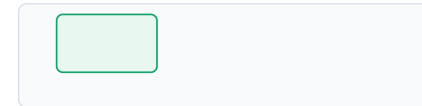
justify-items

по горизонтали внутри ячейки



align-items

по вертикали внутри ячейки



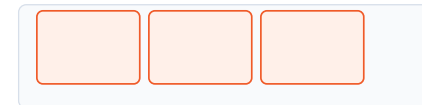
justify-content

вся сетка по горизонтали



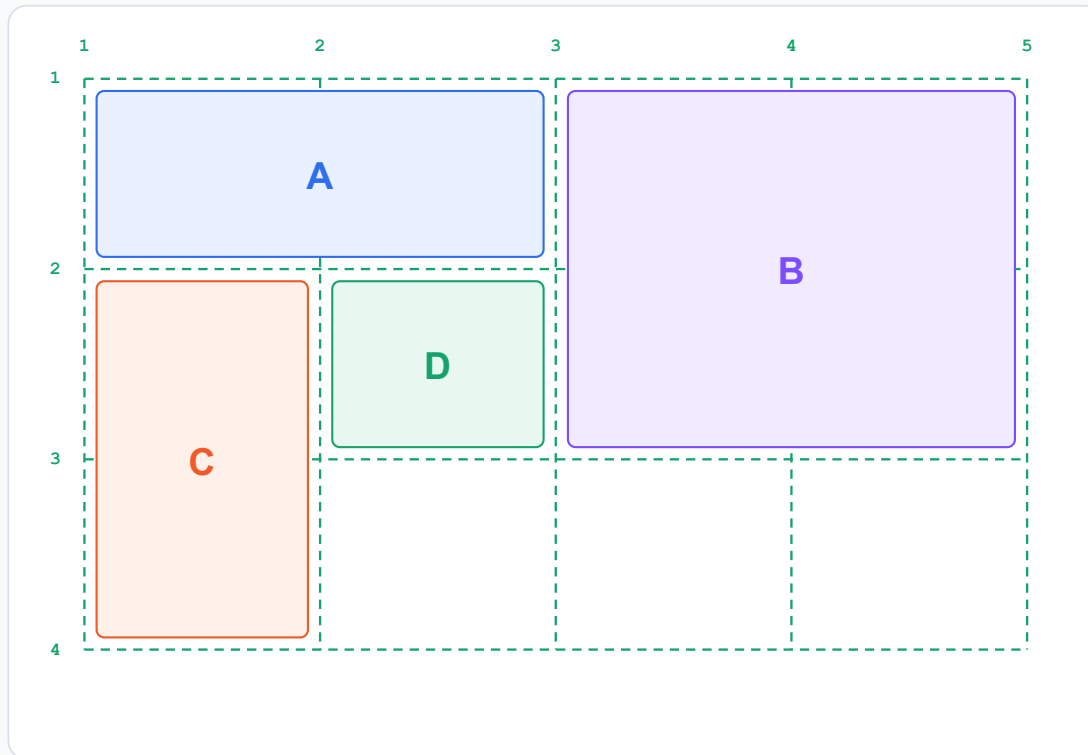
align-content

вся сетка по вертикали



place-items и place-content — сокращения для пары align + justify.

Элементы размещаются между линиями сетки



```
placement.css

.a {
  grid-column: 1 / 3;
}

.b {
  grid-column: 3 / -1;
  grid-row: 1 / span 2;
}

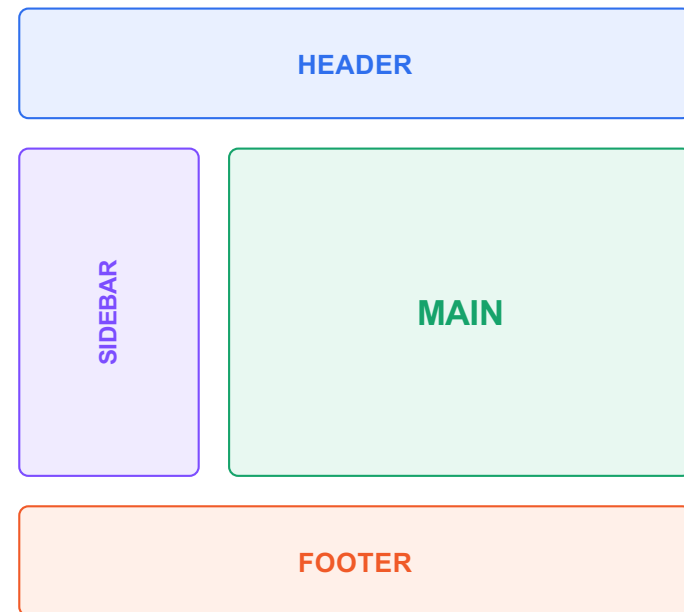
.c {
  grid-row: 2 / 4;
}

.d { grid-column: span 1; }
```

Именованные области читаются как схема страницы

layout.css

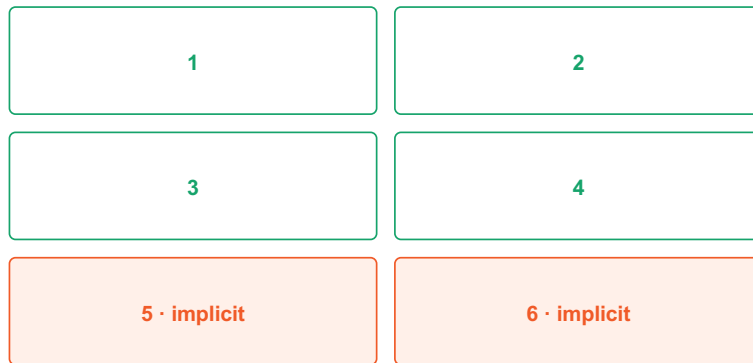
```
.page {  
  display: grid;  
  grid-template-columns: 220px 1fr;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  gap: 16px;  
}  
  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```



Строки в кавычках — визуальная карта, имена связывают items с областями.

Неявная сетка появляется, когда контента больше шаблона

Явно объявлены 2 колонки



третья строка создана автоматически

```
auto.css

.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  grid-auto-rows: 120px;
  grid-auto-flow: row;
}
```

grid-auto-flow: dense

может заполнять «дыры», но визуальный порядок способен отличаться от DOM.

auto-fit растягивает занятые треки, auto-fill сохраняет пустые

cards.css

```
.cards {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit,  
      minmax(220px, 1fr));  
  gap: 20px;  
}
```

AUTO-FILL

Пустые треки остаются

1

2

3

empty

AUTO-FIT

Пустые треки схлопываются

1

2

3

В большинстве галерей карточек auto-fit даёт ожидаемое растягивание.

Собираем адаптивную сетку тарифов

Plans for all sizes

Три тарифа, один рекомендуемый

Basic
990 ₽

Выбрать

POPULAR
Pro
2 490 ₽

Выбрать

Team
4 990 ₽

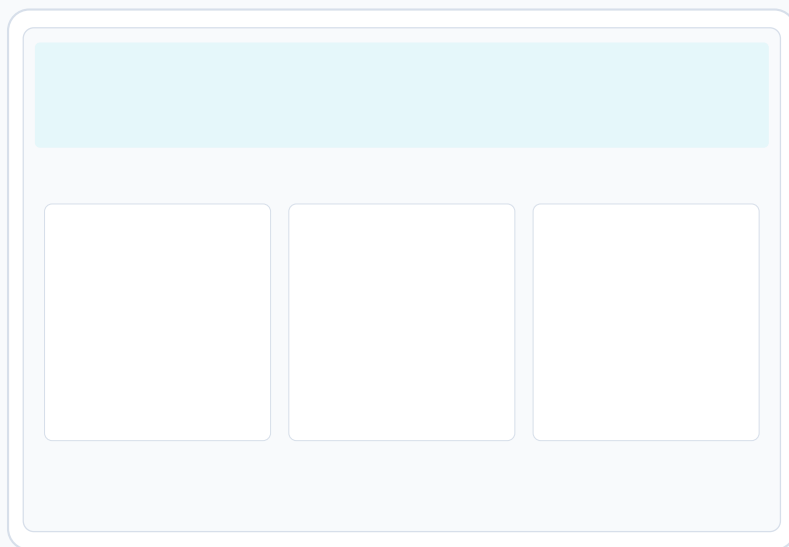
Выбрать

ЗАДАНИЕ

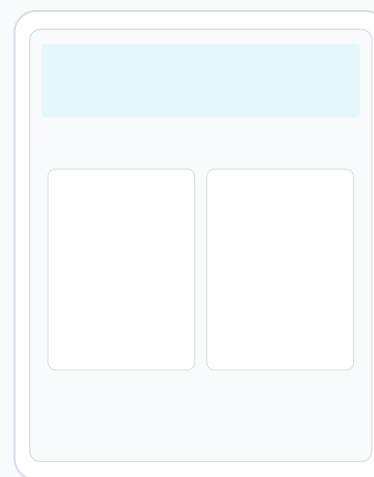
- 1 `display: grid`
- 2 `repeat(auto-fit, minmax(...))`
- 3 gap вместо margin между карточками
- 4 featured-карта только меняет токены

5 МИН

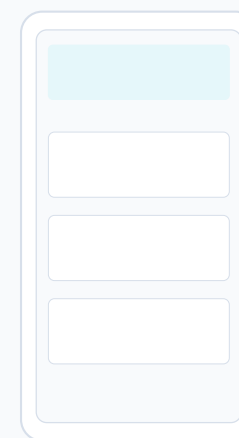
Responsive — это реакция интерфейса на доступное пространство



desktop



tablet



mobile

FLUID

размеры меняются плавно

REFLOW

блоки перестраиваются

PRIORITY

контент остаётся понятным

Сначала viewport и текучая база — потом breakpoints

index.html

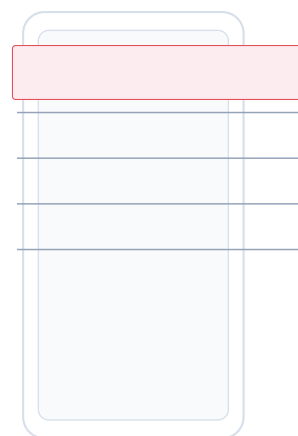
```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
```

base.css

```
.container {
  width: min(100% - 32px, 1200px);
  margin-inline: auto;
}

img { max-width: 100%; height: auto; }
```

Без viewport



страница «ужимается» как desktop

С viewport



CSS-пиксели совпадают с шириной устройства

Fluid-first: проценты, fr, max-width, min()/max()/clamp().

Условие включает набор CSS-правил

responsive.css

```
.cards {  
  grid-template-columns: 1fr;  
}  
  
/* классическая запись */  
@media (min-width: 48rem) {  
  .cards {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
/* range syntax */  
@media (width >= 75rem) {  
  .cards { grid-template-columns: repeat(3, 1fr); }  
}
```

@media (width >= 48rem)

AT-RULE

FEATURE

начало условия

проверка окружения

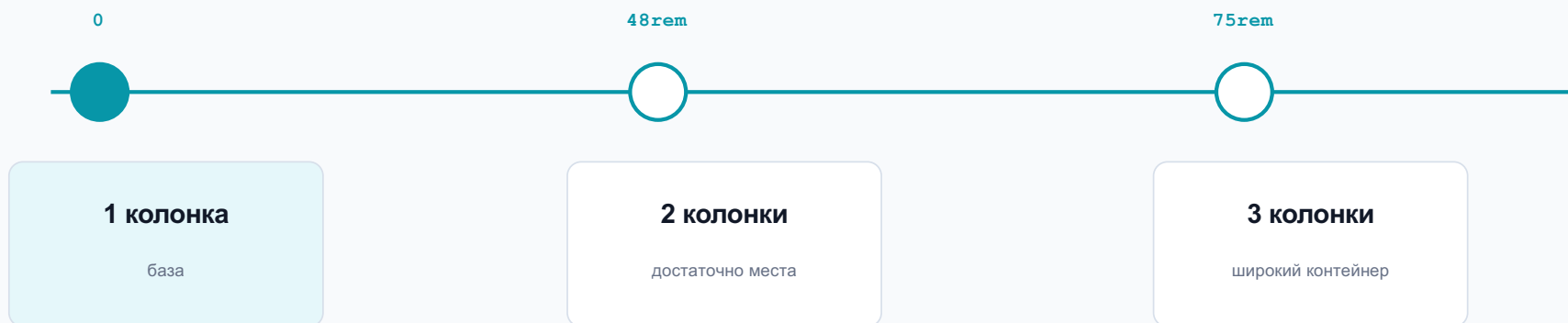
Условие истинно?

ДА

правила участвуют в каскаде

Можно комбинировать: and · not · диапазоны

Mobile-first: базовый интерфейс, затем улучшения



mobile-first.css

```
.cards { grid-template-columns: 1fr; }  
@media (width >= 48rem) { /* enhance */ }  
@media (width >= 75rem) { /* enhance */ }
```

Breakpoints выбирает контент

Не «iPhone / планшет / ноутбук», а момент, когда текущая композиция перестаёт работать.

Относительные единицы и функции уменьшают число breakpoint

% · fr

зависят от контейнера

```
width: 100%  
columns: 1fr 2fr
```

rem · em

связаны с размером шрифта

```
padding: 1.5rem  
line-height: 1.5
```

vw · vh · dvh

связаны с viewport

```
min-height: 100dvh  
font-size: 4vw
```

min · max · clamp

задают безопасный диапазон

```
clamp(1.8rem,  
4vw, 3.6rem)
```

Флюидная типографика

```
font-size: clamp(2rem, 1.2rem + 3vw, 4.5rem);
```

min

fluid preferred

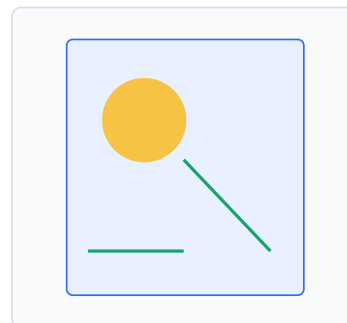
max

Медиа сохраняют пропорции и управляют кадрированием

media.css

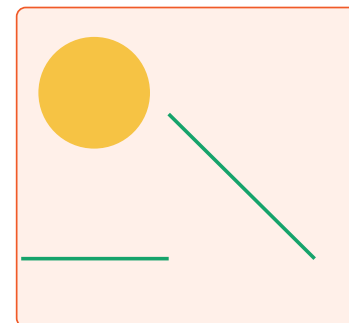
```
img {  
  max-width: 100%;  
  height: auto;  
}  
  
.cover {  
  aspect-ratio: 16 / 9;  
  width: 100%;  
  object-fit: cover;  
  object-position: center;  
}
```

CONTAIN



видно целиком, могут остаться поля

COVER



контейнер заполнен, края могут
обрезаться

Для разных файлов и плотностей экрана HTML добавляет picture / srcset — CSS отвечает за геометрию.

Медиа-функции описывают не только ширину

HOVER

```
@media (hover: hover)
```

показывать hover-эффекты

POINTER

```
@media (pointer: coarse)
```

увеличить touch-target

COLOR SCHEME

```
@media (prefers-color-scheme: dark)
```

переключить токены темы

REDUCED MOTION

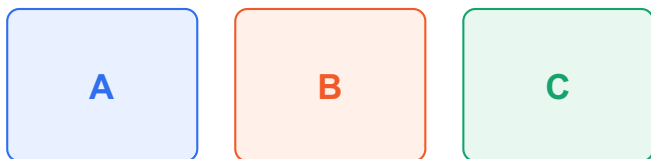
```
@media (prefers-reduced-motion: reduce)
```

убрать лишнее движение

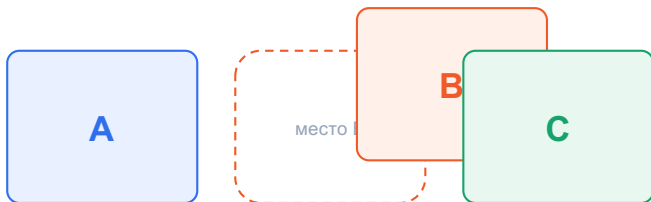
Адаптивность = пространство + ввод + пользовательские предпочтения.

Преобразование меняет картинку, но не нормальный поток

До transform



После transform: translateX(80px)



С не сдвинулся: исходное место В по-прежнему зарезервировано.

motion.css

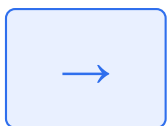
```
.card {  
  transform: translateX(80px);  
}
```

Transform работает после layout

- ✓ не перестраивает соседей
- ✓ может визуально перекрыть контент
- ✓ часто подходит для плавного движения

Функции можно комбинировать — порядок меняет результат

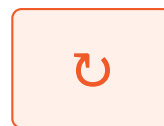
TRANSLATE

`translate(24px, 0)`

SCALE

`scale(1.2)`

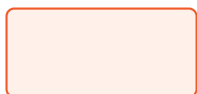
ROTATE

`rotate(15deg)`

SKEW

`skewX(12deg)`

ПОРЯДОК

`translateX(80px) rotate(20deg)` $\neq$ `rotate(20deg) translateX(80px)``transform-origin`

точка вращения

Переход интерполирует значения между двумя состояниями

button.css

```
.button {  
  transform: translateY(0);  
  box-shadow: 0 4px 12px rgb(0 0 0 / .12);  
  transition:  
    transform 180ms ease,  
    box-shadow 180ms ease;  
}  
  
.button:hover {  
  transform: translateY(-2px);  
  box-shadow: 0 10px 24px rgb(0 0 0 / .18);  
}
```

STATE A

Купить



:hover

Купить



0ms



180ms

property

ЧТО МЕНЯЕМ

duration

СКОЛЬКО ДЛИТСЯ

timing

КАК МЕНЯЕТСЯ
СКОРОСТЬ

delay

КОГДА СТАРТУЕТ

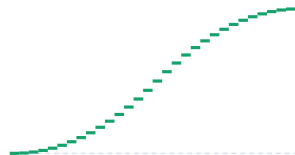
Кривая времени задаёт характер, свойства — стоимость движения

LINEAR



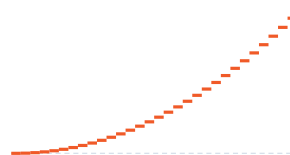
linear

EASE



ease

EASE-IN



ease-in

STEPS



steps (4)

ОБЫЧНО ДЁШЕВО

`transform · opacity`

часто обходятся без полного пересчёта раскладки

ОСТОРОЖНО

`width · height · top · left`

могут затронуть layout и перерисовку соседей

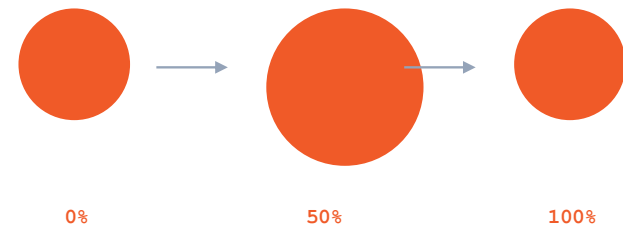
will-change — только после измерения реальной проблемы, а не «на всякий случай».

@keyframes описывает многошаговый сценарий

```
animation.css

@keyframes pulse {
  0% { transform: scale(1); opacity: .7; }
  50% { transform: scale(1.08); opacity: 1; }
  100% { transform: scale(1); opacity: .7; }
}

.status {
  animation: pulse 1.6s ease-in-out infinite;
}
```



`animation: name duration timing delay count direction fill-mode;`

transition — реакция на смену состояния · animation — самостоятельная
временная шкала

Собираем карточку: tokens + Grid + @media + motion

Pricing

POPULAR

Pro

2 490 ₽ / мес

Выбрать

ho



component.css

```
:root { --accent: #7c3aed; --motion: 180ms; }  
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }  
.card { transition: transform var(--motion) ease; }  
.card:hover { transform: translateY(-4px); }
```

REDUCED MOTION

```
@media (prefers-reduced-motion: reduce) {  
  .card { transition: none; transform: none; }  
}
```


Итоги занятия

CSS становится системой: значения → раскладка → условия → движение.

1

Custom properties управляют токенами

каскад, область видимости и темы

2

Grid описывает двумерную раскладку

tracks, areas, placement и auto-fit

3

Адаптивность начинается с fluid layout

@media добавляет изменения по условиям

4

Motion связывает состояния

transform, transition, @keyframes и reduced motion

ПРАКТИКА

Адаптивный блок тарифов

- семантическая HTML-разметка
- токены в :root и локальная тема
- Grid без фиксированного числа колонок
- 1–2 media queries по контенту
- hover + focus-visible + reduced motion

Спасибо! Вопросы?